

Single and Double Inverted Pendulum Case Study

Andrew Shim* Richard Engel*

Abstract

Balancing an inverted single and double pendulum system on a cart is a classical problem in the field of controls and cyber physical systems. Single and double pendulums are examples of nonlinear systems, and the double pendulum specifically is a chaotic system which is more difficult to control. While the control of these systems has been thoroughly studied, the verification of such cyberphysical nonlinear systems is an ongoing field of research, with single and double inverted pendulum controllers being common benchmarks in this area. In this paper, we examine Linear Quadratic Regulator controllers for single and double inverted pendulum systems and implement these controllers in the reachability analysis tool Flow* (3). We then formulate strategies to verify the stability of these controllers using Flow*. We present a straightforward method of completely verifying the single inverted pendulum controller for an initial set, as well as a method for verifying the double inverted pendulum controller. Finally, we present our verification results for both the single and double inverted pendulum controllers.

1. Introduction

The study of nonlinear cyberphysical systems is important due to the place they have in the real world. Cyberphysical systems become increasingly relevant in our lives, from autopilot controls to self-driving cars. The control of cyberphysical systems is an ongoing field of study in its own right, but the verification of such controllers is an area that still has a lot of work to be done. A couple of classic controls examples are the single inverted pendulum and double inverted pendulum cart controllers, which we aim to implement and verify using the tool Flow*. In doing so, we will build a greater understanding of strategies on how to verify nonlinear cyberphysical systems.

*Equal contribution. Correspondence to: Andrew Shim <akshim2@illinois.edu>, Richard Engel <reengel2@illinois.edu>.

Background

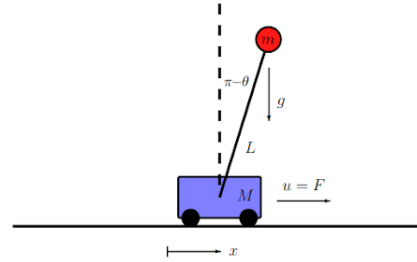


Figure 1. Diagram of SIPC System

Single Inverted Pendulum Controller The Single Inverted Pendulum Controller (SIPC) system we will be using is depicted above (2). The state space consists of four variables: x, v, θ, ω , where x and v are the position and velocity of the cart, θ is the angle of the pendulum arm from the vertical position, and ω is the angular velocity of the pendulum arm. There are also a number of variables we define, such as $M = 1.0$ for cart mass, $m = 0.1$ for mass of the end of the pendulum, $L = 0.5$ for length of the pendulum arm, $g = 9.81$ for the force of gravity, and $\delta = 1$ for the damping coefficient. The state space is governed by the following dynamic equations (2):

$$\begin{aligned} \dot{x} &= v \\ \dot{v} &= \frac{-m^2 L^2 g \cos(\theta) \sin(\theta) + mL^2 \omega^2 (mL \omega^2 \sin(\theta) - \delta v) + mL^2 u}{mL^2 (M + m(1 - \cos(\theta)^2))} \\ \dot{\theta} &= \omega \\ \dot{\omega} &= \frac{(m + M)mgL \sin(\theta) - mL \cos(\theta)(mL \omega^2 \sin(\theta) - \delta v) + mL \cos(\theta)u}{mL^2 (M + m(1 - \cos(\theta)^2))} \end{aligned} \quad (1)$$

The system also includes an input variable u , which will be an input force applied horizontally to the cart. This will be driven by a controller, which will be touched upon later.

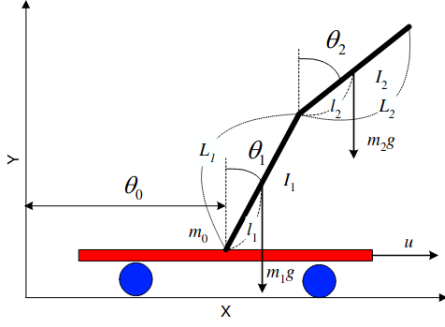


Figure 2. Diagram of DIPC System

Double Inverted Pendulum Controller The Double Inverted Pendulum Controller (DIPC) system we'll be using is shown above (1, 4). This system has a state space that consists of six variables, shown here as $(\theta_0, \theta_1, \theta_2, \dot{\theta}_0, \dot{\theta}_1, \dot{\theta}_2)$. θ_0 and $\dot{\theta}_0$ are the position and velocity of the cart, θ_1 and $\dot{\theta}_1$ are the angle from vertical and angular velocity of the lower arm of the double pendulum, and θ_2 and $\dot{\theta}_2$ are the angle from vertical and angular velocity of the upper arm of the double pendulum. User-defined variables include $m_0 = 1.5$ for the mass of the cart, $m_1 = m_2 = 0.5$ for the masses of the lower and upper arm, $L_1 = L_2 = 0.5$ for the lengths of the lower and upper arms, and $g = 9.8$ for gravitational acceleration. The image also shows variables l_1 and l_2 for the length from base of the center of gravity, and I_1 and I_2 for the moments of inertia for the two arms. To simplify the system though, it is assumed that the center of gravity is at the midpoint of each arm, so these variables don't need to be defined.

The equations of motion (1) for this system can be written in a matrix form:

$$\mathbf{D}(\theta)\ddot{\theta} + \mathbf{C}(\theta, \dot{\theta})\dot{\theta} + \mathbf{G}(\theta) = \mathbf{H}u$$

where

$$\begin{aligned} \mathbf{D}(\theta) &= \begin{pmatrix} d_1 & d_2 \cos(\theta_1) & d_3 \cos(\theta_2) \\ d_2 \cos(\theta_1) & d_4 & d_5 \cos(\theta_1 - \theta_2) \\ d_3 \cos(\theta_2) & d_5 \cos(\theta_1 - \theta_2) & d_6 \end{pmatrix} \\ \mathbf{C}(\theta, \dot{\theta}) &= \begin{pmatrix} 0 & -d_2 \sin(\theta_1)\dot{\theta}_1 & -d_3 \sin(\theta_2)\dot{\theta}_2 \\ 0 & 0 & d_5 \sin(\theta_1 - \theta_2)\dot{\theta}_2 \\ 0 & -d_5 \sin(\theta_1 - \theta_2)\dot{\theta}_1 & 0 \end{pmatrix} \\ \mathbf{G}(\theta) &= \begin{pmatrix} 0 \\ -f_1 \sin(\theta_1) \\ -f_2 \sin(\theta_2) \end{pmatrix} \\ \mathbf{H} &= (1 \quad 0 \quad 0)^T \end{aligned} \quad (2)$$

The coefficients in these matrices are defined as:

$$\begin{aligned} d_1 &= m_0 + m_1 + m_2 \\ d_2 &= \left(\frac{1}{2}m_1 + m_2\right)L_1 \\ d_3 &= \frac{1}{2}m_2L_2 \\ d_4 &= \left(\frac{1}{3}m_1 + m_2\right)L_1^2 \\ d_5 &= \frac{1}{2}m_2L_1L_2 \\ d_6 &= \frac{1}{3}m_2L_2^2 \\ f_1 &= \left(\frac{1}{2}m_1 + m_2\right)L_1g \\ f_2 &= \frac{1}{2}m_2L_2g \end{aligned}$$

From this, we can derive the state space dynamic equations:

$$\dot{\mathbf{x}} = \begin{pmatrix} \mathbf{0} & \mathbf{I} \\ \mathbf{0} & -\mathbf{D}^{-1}\mathbf{C} \end{pmatrix} \mathbf{x} + \begin{pmatrix} \mathbf{0} \\ -\mathbf{D}^{-1}\mathbf{G} \end{pmatrix} + \begin{pmatrix} \mathbf{0} \\ \mathbf{D}^{-1}\mathbf{H} \end{pmatrix} u \quad (3)$$

Again, this system also includes an input variable $[u]$, which is the force applied horizontally to the cart by the controller.

Linear Quadratic Regulator The remaining question is how to design an optimal controller for these systems. One technique is centered around minimizing a cost function:

$$J_t = \sum_{k=t}^{t_{final}} L_k(\mathbf{x}_k, \mathbf{u}_k)$$

This represents the total accumulated cost for every state $\mathbf{x}_k$ and control $\mathbf{u}_k$ from time t to t_{final} . L_k in this case corresponds to Linear Quadratic cost:

$$L_k(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k \quad (4)$$

Here, the $\mathbf{Q}$ matrix represents the cost of deviation from the desired state, and the $\mathbf{R}$ matrix represents the cost of controller input. Both are user-defined. The dynamics equations for SIPC and DIPC can then be linearized around $\mathbf{x}=\mathbf{0}$ as such:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u \quad (5)$$

To implement digital control, matrices $\mathbf{A}$ and $\mathbf{B}$ are approximately discretized into $\Phi \approx e^{\mathbf{A}\Delta t}$ and $\Gamma \approx \mathbf{B}\Delta t$. Optimal digital LQR control is then given by:

$$u_k = -\mathbf{R}^{-1}\Gamma^T \mathbf{P} \mathbf{x}_k \equiv -\mathbf{K} \mathbf{x}_k \quad (6)$$

The $\mathbf{P}$ matrix is found by solving the discrete-time algebraic Riccati equation:

$$\Phi^T [\mathbf{P} - \mathbf{P}\Gamma(\mathbf{R} + \Gamma^T \mathbf{P}\Gamma)^{-1}\Gamma^T \mathbf{P}] \Phi - \mathbf{P} + \mathbf{Q} = \mathbf{0}$$

LQR for SIPC and DIPC For the SIPC, we define $\mathbf{Q}$ and $\mathbf{R}$ as such:

$$\mathbf{Q} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 100 \end{pmatrix}$$

$$\mathbf{R} = 1$$

We then linearize the $\mathbf{A}$ and $\mathbf{B}$ matrices as such (2):

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{\delta}{M} & \frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{\delta}{ML} & -\frac{(m+M)g}{ML} & 0 \end{pmatrix}$$

$$\mathbf{B} = \begin{pmatrix} 0 \\ \frac{1}{M} \\ 0 \\ \frac{1}{ML} \end{pmatrix}$$

This gives us numerical matrices:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & -1 & 0.981 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -2 & 21.582 & 0 \end{pmatrix}$$

$$\mathbf{B} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 2 \end{pmatrix}$$

And from here, we can calculate our $\mathbf{K}$ matrix:

$$\mathbf{K} = \begin{pmatrix} -1.0000 \\ -3.7944 \\ 39.2831 \\ 13.1990 \end{pmatrix}$$

For DPIC, we define $\mathbf{Q}$ and $\mathbf{R}$ as such:

$$\mathbf{Q} = \begin{pmatrix} 5 & 0 & 0 & 0 & 0 & 0 \\ 0 & 50 & 0 & 0 & 0 & 0 \\ 0 & 0 & 50 & 0 & 0 & 0 \\ 0 & 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 700 & 0 \\ 0 & 0 & 0 & 0 & 0 & 700 \end{pmatrix}$$

$$\mathbf{R} = 0.5$$

We can linearize $\mathbf{A}$ and $\mathbf{B}$ as such (1):

$$\mathbf{A} = \begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{D}(\mathbf{0})^{-1} \frac{\partial \mathbf{G}(\mathbf{0})}{\partial \theta} & \mathbf{0} \end{pmatrix}$$

$$\mathbf{B} = \begin{pmatrix} \mathbf{0} \\ \mathbf{D}(\mathbf{0})^{-1} \mathbf{H} \end{pmatrix}$$

This yields:

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & -5.75217 & 0.63913 & 0 & 0 & 0 \\ 0 & 63.1913 & -26.8435 & 0 & 0 & 0 \\ 0 & -80.5304 & 67.7478 & 0 & 0 & 0 \end{pmatrix}$$

$$\mathbf{B} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0.608696 \\ -1.56522 \\ 0.521739 \end{pmatrix}$$

From this, we derive our $\mathbf{K}$ matrix:

$$\mathbf{K} = \begin{pmatrix} 3.0385 \\ -589.2321 \\ 684.6339 \\ 11.0901 \\ -13.5135 \\ 95.9474 \end{pmatrix}$$

The problem Our main goal is to take these LQR controller designs for the SIPC and DIPC systems and verify their stability. For the SIPC, we will consider an initial set:

$$X_0 = \{\mathbf{x} \in \mathbb{R}^4 : x = 0, v = 0, \theta \in [-0.5, 0.5], \omega \in [-1.0, 1.0]\} \quad (7)$$

We want to verify that SIPC stays within the safe set S_{SIPC} over time horizon $T = 5.0$ seconds:

$$S_{SIPC} = \{x \in \mathbb{R}^4 : -2 \leq x \leq 2 \wedge -\frac{\pi}{2} \leq \theta \leq \frac{\pi}{2}\} \quad (8)$$

$$\forall \mathbf{x}_0 \in X_0, \forall t \in [0, T] : \phi(\mathbf{x}_0, t) \in S$$

For the DIPC, we must take a different approach for reasons that will be discussed later. We will verify the range of safe initial angles and angular velocities for a time horizon of $T = 4.0$ seconds. We will use single initial states rather than initial set intervals, and will combine these results to create a verified safe set. We will do this using a manual binary search with a granularity of 0.01 seconds, and we will consider the system test safe if it doesn't terminate early due to overapproximation, which is our safety condition S_{DIPC} :

$$X_0, \forall t \in [0, t] : S_{DIPC} \quad (9)$$

State of the art There has been much work done in the study of single and double inverted pendulum control and the control of nonlinear systems in general, and there is ongoing research into the verification of cyberphysical nonlinear controllers. As the controllers get more complex, it becomes necessary to combine and extend the functionality

of tools rather than using a singular tool. This case study serves as an introduction to analyzing nonlinear cyberphysical systems using tools such as Flow*, and it serves as a gateway into verification research for complex cyberphysical systems using a multitude of verification tools.

Our approach We plan to achieve these verifications by using Flow*. Flow* is a tool that performs reachability verification by utilizing Taylor Models to generate flowpipes (3). Flowpipes are essentially overapproximations of the states that would be reachable from the initial set, guaranteeing that the reachable states are contained within these flowpipes. If a flowpipe completely enters a user-defined unsafe set, the system is declared unsafe. If a flowpipe completely avoids the unsafe set, the system is declared safe. A flowpipe partially intersecting the unsafe set is an unknown result.

Our approach was to first implement the SIPC into Flow* to get used to using the tool. We first implemented the dynamics of the system, and then implemented the control law. Then we attempted a direct reachability analysis on the system with a defined initial set. Once this was achieved, we then implemented the DIPC by first implementing the dynamics and then the control law, and attempted reachability analysis on the system using single initial states rather than initial sets. For both systems, our strategy was to attempt direct reachability analysis first, but the increased complexity of the DIPC necessitated a different approach that will be explained in the Methodology section.

Contributions Our case study into the SIPC and DIPC systems got us more familiar with types of classical control and how to work with those controllers in Flow*. Having grown comfortable with using Flow* to implement them, this opens the doorway into further experimentation with how to verify different cyberphysical systems, how to implement different types of controllers, and how to extend Flow* and potentially use it with other tools for a more complete verification of complex controllers.

2. Related work

Extensive study has been done previously into the control of both SIPC and DIPC systems, using a variety of classical controllers. The motion of a single pendulum is of course a classic example of simple harmonic motion, while the double pendulum is a classic example of chaotic motion. The exact regions of stability concerning the double pendulum have also been analysed (5), which allows for the control of such a system. Significant work has been done towards the design of LQR controllers for both the single pendulum (2) and double pendulum (4, 1) systems, and there has also been work done in further controller types for these systems such

as a State Dependent Riccati Equation (SDRE) controller (1). From this, it's clear that the design of controllers for these systems is fairly solid.

Extensive work has also been done in developing the Flow* tool to work with cyberphysical nonlinear systems such as these (3). Further studies have been done on extending Flow* to be used with other tools such as α, β -CROWN for the verification of neural-network controlled nonlinear systems (6), with research ongoing for this area. Our goal is to take a step back and directly implement classical controller designs for the SIPC and DIPC systems in Flow* and to see what we can do with that. This focused case study will give us a solid foundation to launch into future studies concerning the verification of nonlinear cyberphysical systems.

3. Methodology

SIPC Implementation Implementing the SIPC system in Flow* is fairly straightforward. Flow* provides the functions and classes necessary to both incorporate the ODEs and the control law of the system and build a testing routine in C++. The first step is to create a Flow* Variables object and to define the necessary variables for the system. This includes both the state space variables and the input variables. For SIPC, we have four state space variables and one input variable, assigned as such:

```
// Declare variables
Variables vars;
int x_id = vars.declareVar("x");
int v_id = vars.declareVar("v");
int theta_id = vars.declareVar("theta");
int omega_id = vars.declareVar("omega");
int u_id = vars.declareVar("u");
```

Figure 3. Declaring variables for SIPC

The next step is to push the ODEs of the system into a vector of strings. The order of the ODEs in the string vector must correspond to the order that the variables were assigned. For the SIPC, we incorporate the ODEs from 1 like so (note, user-defined parameters are written in directly for this case):

```
vector<string> ode = {"v",
    "(-0.01*0.25*9.81*cos(theta)*sin(theta) + 0.1*0.25*(0.1*0.5*omega^2*sin(theta) - v) + 0.1*0.25*u) / (0.1*0.25*(1.0 + 0.1*(1 - cos(theta)^2)))",
    "omega",
    "(1.1*0.1*9.81*0.5*sin(theta) - 0.1*0.5*cos(theta)*(0.1*0.5*omega^2*sin(theta) - v) + 0.1*0.5*cos(theta)*u) / (0.1*0.25*(1.0 + 0.1*(1 - cos(theta)^2)))"};
```

Figure 4. Defining ODEs for SIPC

For systems with controller input, it is also necessary to define the control laws into its own vector of strings. In our case, since we only have one controller input variable, we only need to define one control law. We incorporate 6 and the associated K values like so:

```
// LQR controller gains
double K1 = -1.0000;
double K2 = -3.7944;
double K3 = 39.2831;
double K4 = 13.1990;

vector<string> ctrlLaw = {"(-" + to_string(K1) + "*x - "
    + to_string(K2) + "*v - "
    + to_string(K3) + "*theta - "
    + to_string(K4) + "*omega)"};
```

Figure 5. Declaring control law for SIPC

Finally, we can create a Flow* Feedback object with the defined variables, ODEs, control laws, and a user-assigned control step size. For the SIPC, we chose a step size of 0.01 seconds.

```
Feedback<Real> feedback(vars, 0.01, ode, ctrl_low);
```

Figure 6. Initializing Feedback object for SIPC

With the feedback object created, we can then create a `Flow*Computational.Setting` object, which we can use to set our step size, order, and a maximum remainder estimation for this system. This also makes use of `Flow*`'s `Interval` class. `Flow*` allows the user to set either an adaptive step size or an adaptive TM order to help ensure that the error at each step stays within the maximum remainder interval (3). The initial set is also defined using `Intervals` and assigned to a vector, and the safe set is defined using a vector of `Flow*Constraints`. A time horizon must also be defined, along with a `Flow*Symbolic.Remainder` object.

Once all of these parameters have been initialized and set, we can run the reachability analysis on the Feedback object. The result and Taylor Models from the reachability analysis is then stored in a Flow* *Reach_of_Reachability* object, and the result can then be analyzed using built in functions such as *isCompleted()* and *isSafe()*. At this point the flowpipes can also be plotted by using the *transformToTaylorModels()* function on the results, and by creating a Flow* *Plot_Setting* object. After this, the plots can be output to plot files for user-selected variable dimensions.

With all of this in place, we can directly set our desired initial set and safe set and run a direct reachability analysis with those parameters. This works well for the SIPC, but

taking such a direct approach for the DIPC leads to some issues, which will be discussed in the next section.

DIPC Implementation and Issues The DIPC system implementation in Flow* takes the same fundamental approach as the SIPC system. Due to the complexity of the ODE, we started by defining intermediary strings d1-d6 and f1-f2, corresponding to 2. This is shown in Figure 7.

```
// System parameters
double M = 1.5; // mass of cart in kg
double m1 = 0.5; // mass of first arm in kg
double m2 = 0.5; // mass of second arm in kg
double l1 = 0.5; // length of first arm in meters
double l2 = 0.5; // length of second arm in meters
double g = 9.8; // gravitational acceleration in m/s^2

// Matrix coefficient expressions
string d1 = to_string(M + m1 + m2);
string d2 = to_string((0.5*m1 + m2)*L1);
string d3 = to_string(0.5*m2*L2);
string d4 = to_string(((1.0/3.0)*m1 + m2)*L1*L1);
string d5 = to_string(0.5*m2*L1*L2);
string d6 = to_string((1.0/3.0)*m2*L2*L2);

string f1 = to_string((0.5*m1 + m2)*L1*g);
string f2 = to_string(0.5*m2*L2*g);
```

Figure 7. Defining DIPC coefficient strings

We then used these coefficients to build strings for every non-zero entry in the matrices used in 2, as shown in Figure 8.

```
// Define ODES
string D_01 = "((-d1+***d4+***d6+* - +d2+***2*cos(theta1)^2 - +d1
+***+d5+***2*cos(theta1-theta2)^2 - +d2+***+d3+***+d5+***cos(theta1)*cos
(theta1-theta2)*cos(theta2) - +d3+***2+***d4+***cos(theta2)^2));";
string D_11 = "((-+d0_c+***+d4+***+d6+* - +d5+***2*cos(theta1-theta2)^2));";
string D_12 = "((-+d0_c+***(-+d2+***+d6+***cos(theta1) + +d3+***+d5+***cos
(theta1-theta2)*cos(theta2))))";
string D_13 = "((-+d0_c+***(+d2+***+d5+***cos(theta1)*cos(theta1-theta2) - +d3
+***+d4+***cos(theta2))))";
string D_21 = D_12;
string D_22 = "((-+d0_c+***+d1+***+d6+* - +d3+***2*cos(theta2)^2));";
string D_23 = "((-+d0_c+***(-+d1+***+d5+***cos(theta1-theta2) + +d2+***+d3+***cos
(theta1)*cos(theta2))))";
string D_31 = D_13;
string D_32 = D_23;
string D_33 = "((-+d0_c+***+d1+***+d4+* - +d2+***2*cos(theta1)^2));";

string C_12 = "((-+d2+***sin(theta1)*omegal1);";
string C_13 = "((-+d3+***sin(theta2)*omegal2);";
string C_23 = "((-+d5+***sin(theta1-theta2)*omegal2);";
string C_32 = "((-+d5+***sin(theta1-theta2)*omegal1);";

string G_21 = "((-+f1+***sin(theta1));";
string G_31 = "((-+f2+***sin(theta2));";
```

Figure 8. Defining DIPC non-zero matrix members

The final step is to use these matrix entries to expand 3 out to the individual state space ODEs, shown in Figure 9:

```
vector<string> ode = {"v", "omega1", "omega2"};
ode.push_back("omega1*(-^D 11+****C 12+****C 13+****C 32+*) + omega2*(-^D 11  
+****C 13+****C 12+****C 23+*) + u*(-^D 11+*) + (-^D 12+****G 21+****D 13+****  
+G 31+*)");
ode.push_back("omega2*(-^D 21+****C 12+****D 23+****C 32+*) + omega2*(-^D 21  
+****C 13+****D 22+****C 23+*) + u*(-^D 21+*) + (-^D 22+****G 21+****D 23+****  
+G 31+*)");
ode.push_back("omega1*(-^D 31+****C 12+****D 33+****C 32+*) + omega2*(-^D 31  
+****C 13+****D 32+****C 23+*) + u*(-^D 31+*) + (-^D 32+****G 21+****D 33+****  
+G 31+*)");
```

Figure 9. Setting up the state space ODEs

Aside from these steps, the setup is largely the same. We define the control law, create the Feedback object (for DIPC we picked a control step of 0.001 seconds), and define the necessary parameters for reachability analysis. For the DIPC system, due to its chaotic nature, we decided to use single initial states rather than initial sets for analysis.

With this implementation of DIPC in Flow* however, we noticed some undesired results compared to the SIPC. The simulation exits with the message “Flowpipe computation terminated due to large overestimation”. When the flowpipe overapproximation error accumulates over time, past a *cut-off-threshold* value defined within the Flow* tool, then this message is printed and the simulation is terminated. This error accumulation behavior is clearly visible in the output taylor flowpipe plots.

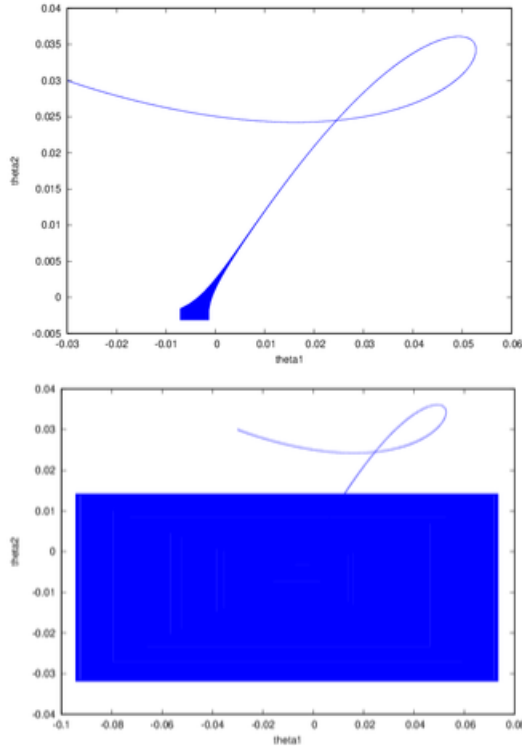


Figure 10. Flowpipe trajectories. The top graph shows the trajectory beginning to blow up, and the bottom graph shows a further overestimation.

This overapproximation “blowup” is largely unavoidable for the DIPC model due to its complex nature. Even very similar initial points will result in drastically different execution paths, diverging more and more as time increases. Because the flowpipe polygon will have to be drawn to encompass all possible execution paths, the diverging nature will quickly lead to blowups. This eliminates our ability to define an initial set for DIPC simulations. Instead we start from the

smallest possible set: a point. However, even from a point the simulation overapproximation still exists, since Flow* automatically sets a tiny interval around the initial point.

There are a few strategies one can use to minimize the overapproximation blowup as much as possible. Higher order of Taylor series approximations and smaller stepsizes of Flow* simulations both act to improve the accuracy of the simulation and thus reduce the overapproximation error. This approach does delay the blowing up behavior, but it isn’t eliminated; moreover, the computation time increases very quickly in an exponential manner.

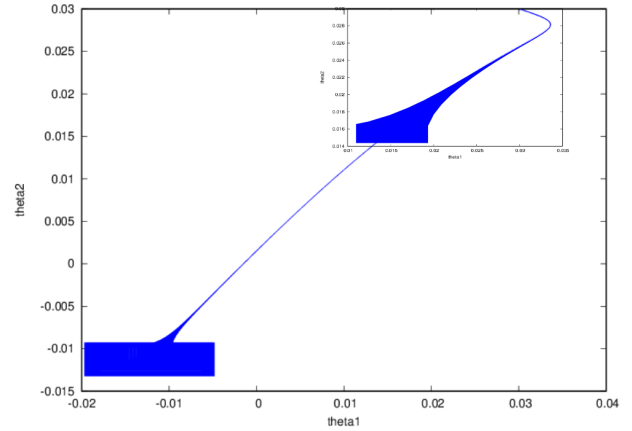


Figure 11. An overlay of two runs from the same initial state, showing the effect of a higher TM order.

Verification Strategy Based on our preliminary testing we developed the following methodology to complete our verification. The double pendulum model is limited to verification runs within $T = 4s$. This timeframe allows most models with Taylor order 3 to complete without overapproximation issues. These simulations complete in about 4-6 minutes, and serve as a good tradeoff between simulation accuracy and computation cost.

Since we cannot define a larger initial set, we run a reachability check on a finite set of distinct and representative points within the wider initial set. We define an initial point as a verified safe initial point for the DIPC if the simulation is able to complete without “termination due to large overestimation”.

Across our entire test set, there was no run that completed successfully where the feedback controller was unable to bring the pendulum arms towards the target upright position. This being the case, there was no need to define any safe set, or implement some check around the pendulum arm positions. The above condition of successful program completion was sufficient. This behavior will be further

explained in the results section of this report.

4. Results

Benchmarks and models. The LQR controller punishes heavily for deviations from the target angles and angular velocities. As such, we will focus on a reachability analysis on a set of ranging initial pendulum arm angles and pendulum angle velocities. The verification of the SIPC system is straightforward due to the much faster runtime and lower complexity. The DIPC system is more complex however, so we take the alternate approach discussed previously. We test both equal initial angles and differing initial angles, and we test initial angular velocities separately. As discussed before, we will test a set of initial points rather than initial sets due to the tendency of the simulation to blow up.

Evaluation metrics. As discussed before, our criteria for safety is based on the completion of the simulation up to $T = 4.0$ seconds. Every run that was successfully completed, we observed that the controller successfully brought the system back towards equilibrium, which is a strong indicator of stability given how sensitive a double pendulum system is and how difficult it would be to keep stable. Nearly every run that didn't successfully finish due to overestimation indicates an initial state where the controller's motion is likely to deviate over the long term, which seems to be a decent indicator of instability.

The only exception to this is the initial state of $\mathbf{x} = 0$, or equilibrium. In this case, the simulation ends early due to overestimation, but this is just due to a quirk of how Flow* works. A single initial state is converted to a tiny initial set surrounding this point state during the reachability analysis. Therefore, with an initial state at equilibrium, you have possible starting states with both slightly negative and slightly positive angles and angular velocities. These initial states would send the cart controller in opposite directions, quickly expanding the flowpipe overestimation from the initial set, and the limitation of our TM order to 3 leads to an early simulation failure. This case is an outlier, as all initial states we tested that were close to equilibrium showed strong stability with minimal overestimation in the outputs.

SIPC Results As discussed earlier, it is straightforward to run the reachability analysis on the SIPC system with our desired initial set. The figures below show the plotted output given our initial set as defined in 7.

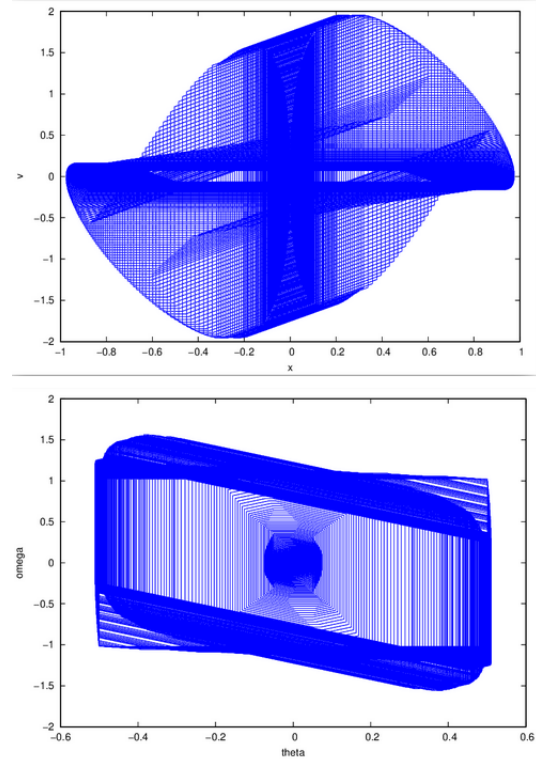


Figure 12. Flowpipe graphs. Top is v vs x , bottom is ω vs θ .

These graphs represent an overestimation of the possible position, velocity, angular, and angular velocity values that the SIPC can have starting with the initial set. In the upper graph, the flowpipe starts at $x, v = 0$, but both values must move outward to rebalance the pendulum, with the velocity eventually settling near 0. With a longer time horizon, the position of the cart would eventually also return to 0. In the right graph, the flowpipe starts at the initial set, and over time converges to $\theta, \omega = 0$. These graphs are pretty hard to read, so we can split apart the initial set to make the motion clearer.

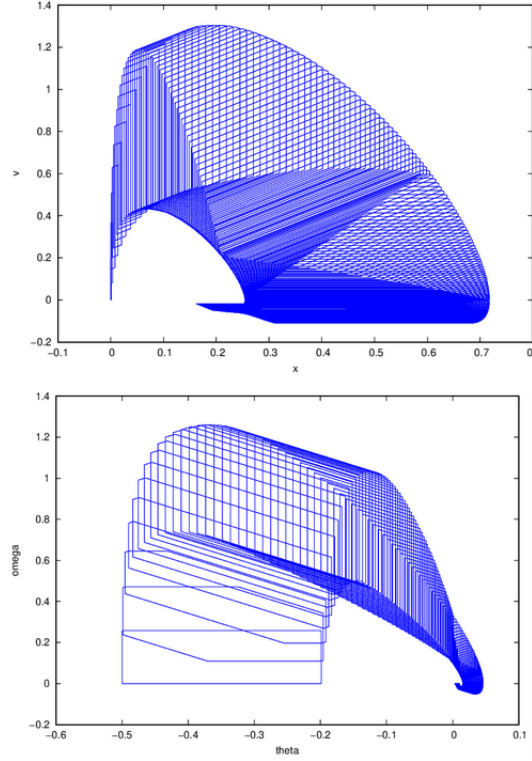


Figure 13. Flowpipe graphs for $\theta_0 \in [-0.5, -0.2]$

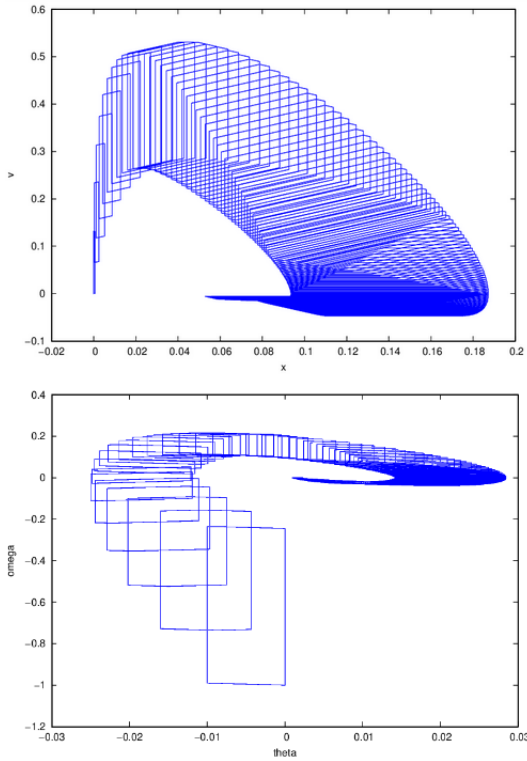


Figure 14. Flowpipe graphs for $\omega_0 \in [-1.0^{-1}, -0.5^{-1}]$

Figure 13 depicts the state space motion with only an outer slice of the initial angular set, and Figure 14 depicts the state space motion with an outer slice of the initial angular velocity set. The upper graph for both figures show a similar cart motion over the time horizon, with the position and velocity flowpipe starting at around 0, expanding outward and over to the right of the graph, and converges to $v = 0$ near the time horizon. In both lower graphs, it's clear to see how the flowpipe starts at the defined initial set and converges to $\theta, \omega = 0$ over time. From all of this, the safety of the single inverted pendulum controller with the initial set 7 is verified for time horizon $T = 5.0$ seconds.

DIPC - Same Angles We wanted to test a range of different conditions for the DIPC system, so first we decided to run some tests with $\theta_1^{(0)} = \theta_2^{(0)}$. We ran lots of test runs with initial angles between -0.8 and 0.8 , with a few of the resulting graphs being shown here.

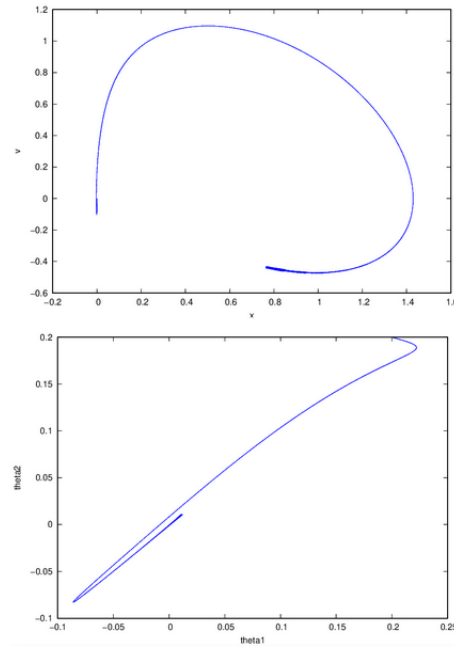


Figure 15. Flowpipe graphs for $\theta_1^{(0)} = \theta_2^{(0)} = 0.2$

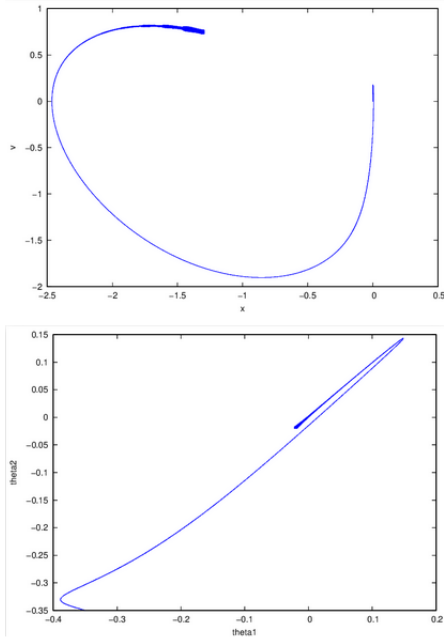


Figure 16. Flowpipe graphs for $\theta_1^{(0)} = \theta_2^{(0)} = -0.35$

We can see from the above figures that the system follows a similarly shaped trajectory for the angles in this range. The overall domain of values varies depending on the size of the initial angle, and the motion for negative and positive angles are diagonally mirrored, but the overall shape of the trajectories are very similar.

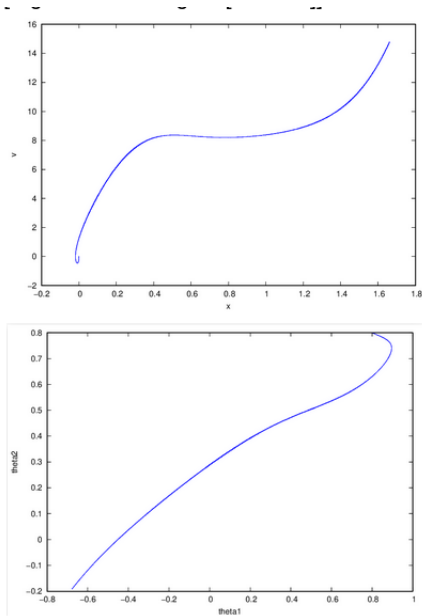


Figure 17. Flowpipe graphs for $\theta_1^{(0)} = \theta_2^{(0)} = 0.8$

The above graphs depict a failure, where the simulation ended very early on due to overestimation. The cart position and velocity begin to take off, and it fails to bring the angles closer to 0 before the simulation ends. From further testing, we concluded that the controller is safe with an initial angle range of $[-0.70, 0.70]$ up to a time horizon of $T = 4.0$ seconds, according to our criteria.

DIPC - Differing Angles In addition to straight-arm angles, we wanted to test how the system would behave with differing initial angles. The angles themselves we kept within -0.5 and 0.5 radians, since we've already established that to be well within safety for our purposes. One of the successful test runs is shown in Figure 18.

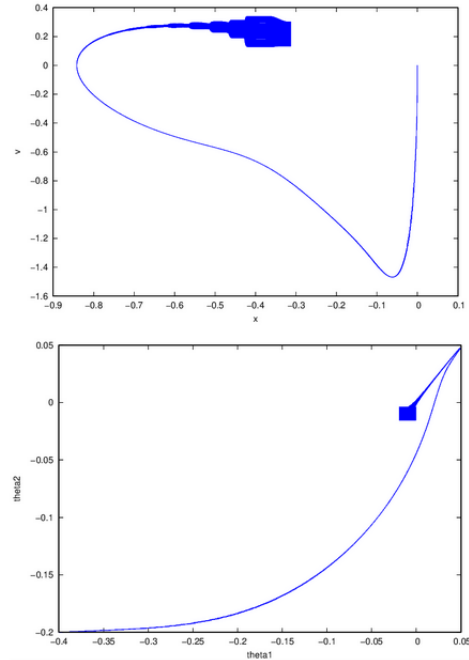


Figure 18. Flowpipe graphs for $\theta_1^{(0)} = -0.4, \theta_2^{(0)} = -0.2$

Through our testing, we've found that the stability of the system with differing initial angles is largely dependent on the size of the difference between $\theta_1^{(0)}$ and $\theta_2^{(0)}$, as well as the magnitude of $\theta_1^{(0)}$. An increase of either value leads to a decrease in stability, and a larger $\theta_1^{(0)}$ decreases the threshold of the difference between the angles before the system becomes unstable.

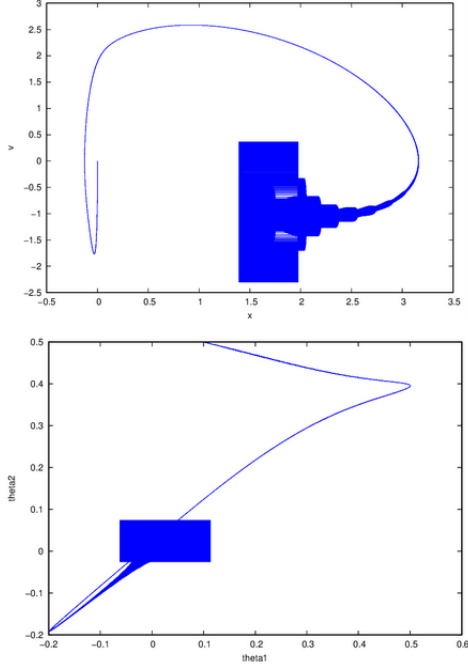


Figure 19. Flowpipe graphs for $\theta_1^{(0)} = 0.1, \theta_2^{(0)} = 0.5$. Failure

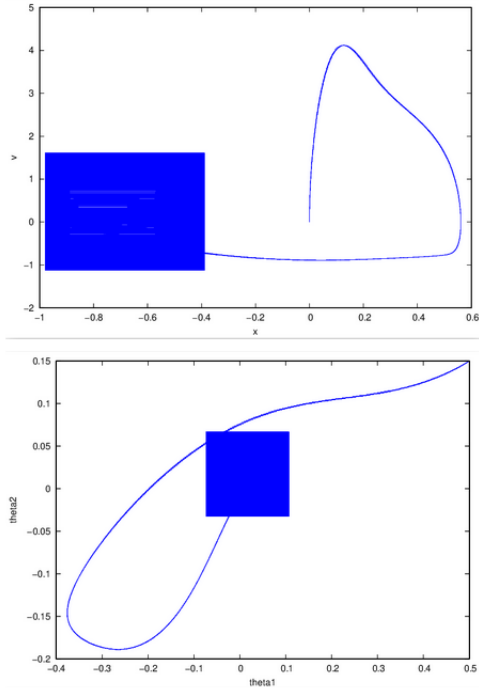


Figure 20. Flowpipe graphs for $\theta_1^{(0)} = 0.5, \theta_2^{(0)} = 0.15$. Failure

Figures 19 and 20 showcase two examples of failure. The first figure has a smaller value of $\theta_1^{(0)} = 0.1$, and the second

figure has a larger value of $\theta_1^{(0)} = 0.5$. In the former case, the system very nearly made it to the time horizon, falling short by just a few control steps. The latter case was more unstable, stopping before reaching even 3.5 seconds. Indeed, we found through thorough testing that the maximum safe difference between the two initial angles decreases as $\theta_1^{(0)}$ increases. For these two cases, we found that the maximum safe difference if $\theta_1^{(0)} = 0.1$ is about 0.39, while the safe difference if $\theta_1^{(0)} = 0.5$ is only 0.26. From this, we conclude that the maximum safe difference for this system up to $T = 4.0$ seconds is 0.26, provided that $-0.5 \leq \theta_1^{(0)}, \theta_2^{(0)} \leq 0.5$

DIPC - Differing Angular Velocities For the purposes of verifying the safe initial set of angular velocities, we start each verification with both pendulum arm angles at 5° from center. We took a binary-search like approach in order to find an accurate value for the maximum safe angular velocities in either direction.

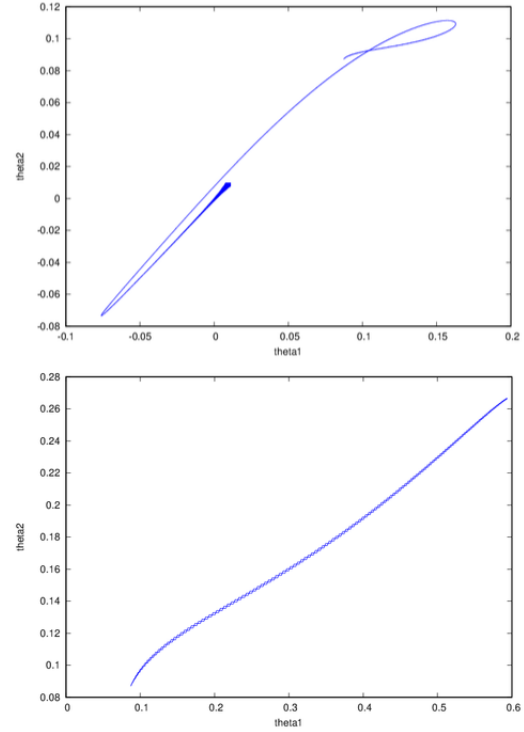


Figure 21. Two flowpipe trajectories from separate angular velocity tests. The top shows a successful run, the bottom shows a failure.

The two plots show the trajectory of the double inverted pendulum in terms of θ_1 and θ_2 . Both runs have identical $\theta_1 = \theta_2 = 0.0873$ (5°), but with different positive initial angular velocities. At $\dot{\theta} = 1.28s^{-1}$ the controller is verified safe. Following the trajectory of the model, note the maximum positive angle deflection in θ_1 of about 0.16. The pendulum arm then swings back towards upright, overshoots

to a maximum negative angle of about -0.075 , before the verification completes near the upright angle. This controller is exhibiting damped oscillatory behavior, which is desired.

In contrast, with $\dot{\theta} = 2.08s^{-1}$, the controller fails to suppress the large initial velocity and swing the pendulum arms back toward 0. Similarly to the results from the safe angle verification, we also saw initial points near the boundary between safe and unsafe that terminate early before $T = 4s$, even though it exhibits similar damped oscillatory behavior.

In total, 16 separate runs were completed between the range of $[-4, 2.56]$. Within this range, we successfully verified the following subset between $[-1.76, 1.44]$ as safe.

In all of the simulations above, the control force is limited by the minimized cost function. In certain applications of inverted pendulum verification, a maximum force would be desirable, and this is possible within Flow*. For the purposes of this paper, we explored the full capabilities of a LQR controller outside of arbitrary physical constraints.

Code. We have stored all of our code in the github repository linked below. The code for the SIPC and DIPC systems are found within the *flowstar_models* folder.

[https://github.com/andykshim/
Inv-Pendulum-Verification/tree/main](https://github.com/andykshim/Inv-Pendulum-Verification/tree/main)

References

- [1] Alexander Bogdanov. Optimal control of a double inverted pendulum on a cart. Technical Report CSE-04-006, Oregon Health & Science University, Portland, Oregon, December 2004.
- [2] Steven L. Brunton and J. Nathan Kutz. *Linear Control Theory*, pages 352–353. Brunton & Kutz, 2017.
- [3] Xin Chen, Erika Abraham, and Sriram Sankaranarayanan. Flow*: An analyzer for non-linear hybrid systems. In Natasha Sharygina and Helmut Veith, editors, *Computer Aided Verification*, Lecture Notes in Computer Science, pages 258–263, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [4] Ian Crowe-Wright. Control theory: Double pendulum inverted on a cart. Master’s thesis, The University of New Mexico, Albuquerque, New Mexico, December 2018.
- [5] Javier Jimenez-Lopez and Victor J. Garcia-Garrido. Chaos and regularity in the double pendulum with lagrangian descriptors. *International Journal of Bifurcation and Chaos*, 34(16), 2024.
- [6] Diego Manzananas Lopez and et al. Arch-comp24 category report: Artificial intelligence and neural network

control systems (ainnecs) for continuous and hybrid systems plants. In G. Frehse and M. Althoff, editors, *11th International Workshop on Applied Verification for Continuous and Hybrid Systems*, volume 103 of *EPiC Series in Computing*, pages 64–121, 2024.